Terraform for AWS

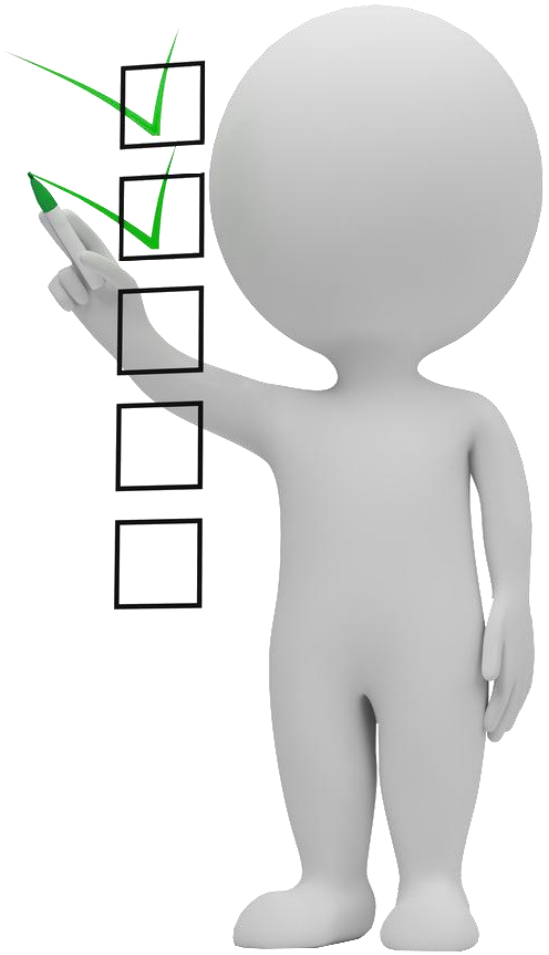
모듈 02 – Terms and Concepts

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

MEGAZONE CLOUD

Objectives

01	Terraform workflow 설명
02	Terraform 내에서 기본 구성 파일의 구조 이해
03	Terraform 몇 가지 명령어의 용도 이해
04	Terraform 유효성 검사기 도구 이해
05	실습 : Terraform 사용하여 AWS Cloud 리소스를 생성, 업데이트 및 삭제



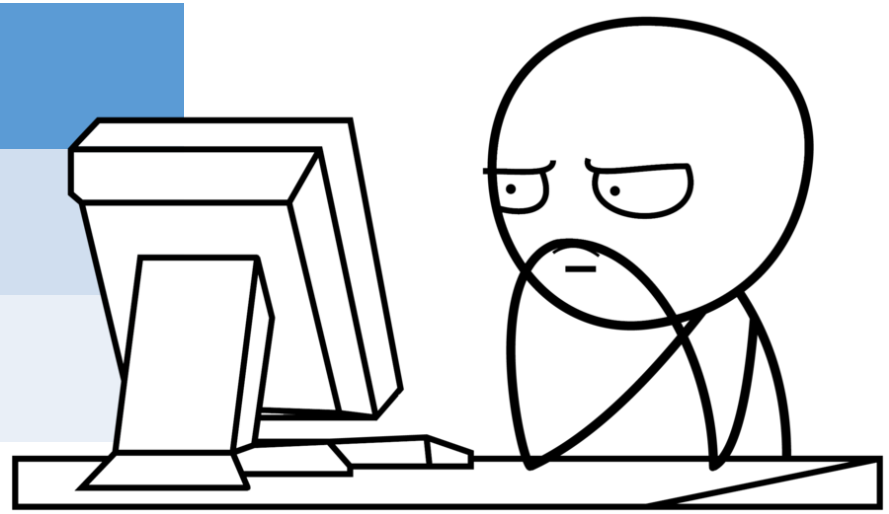
Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 The Author phase(작성 단계)

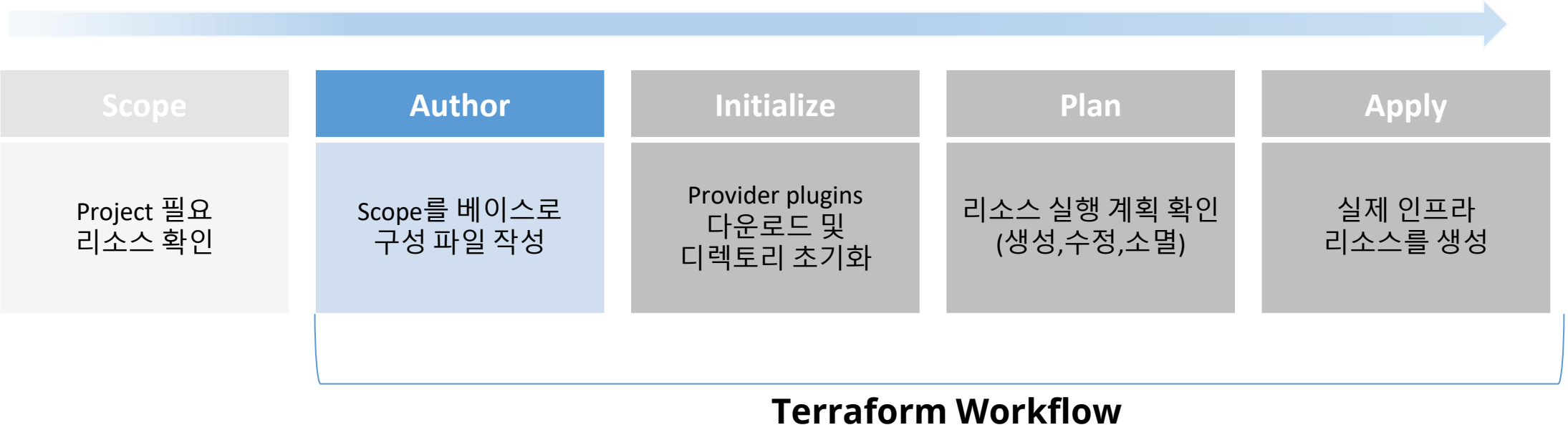
02 Terraform commands

03 Terraform Validator Tool



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Terraform workflow



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Terraform directory

- Terraform은 구성파일을 이용하여 인프라 요소를 선언
- 구성 파일의 확장자가 .tf인 테라폼 언어로 작성
- Terraform은 다음 방법으로 구성 파일 작성
  - ✓ A root module/root configuration
  - ✓ Zero or more child modules
  - ✓ Variable.tf (optional but recommended)
  - ✓ Outputs.tf (optional but recommended)
- Terraform 명령은 작업 디렉토리(Root moudle)에서 실행

```
-- main.tf
    -- servers /
        -- main.tf
        -- providers.tf
        -- variables.tf
        -- outputs.tf
```

Root module

Child module

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

HashiCorp Configuration Language(HCL)

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> #Argument
}
```

- **API-based 리소스를 생성하고 관리하기 위한 Terraform 구성 언어**
- **프로그래밍 언어가 아닌 구성 언어(JSON 기반 변형)**
- **variables, resources, outputs 및 modules 같은 제한된 기본 요소 집합을 포함**
- **전통적인 명령문 또는 제어 루프를 포함**

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

HCL syntax

Blocks | Arguments | Identifilers | Expressions | Comments

```
resource “aws_instance" “example" {
  # A custom mode EC2 instance resource
  instance_type            = “t2.micro”
  ami                      = “ami-zbc123ef”

  tags = {
    Name                   = “demo-ec2”
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

HCL syntax

Blocks | Arguments | Identifilers | Expressions | Comments

Block은 특정 유형에 속하는 코드 줄입니다. 블록의 몇 가지 예로는 리소스, 변수, 출력이 있습니다.

블록은 단순하거나 다른 블록 유형을 포함도록 중첩될 수 있습니다.

```
resource “aws_instance" “example" {
  # EC2 instance resource
  instance_type                    = “t2.micro”
  ami                              = “ami-zbc123ef”

  tags = {
    Name = “demo-ec2”
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

HCL syntax

Blocks | **Arguments** | Identifilers | Expressions | Comments

Arguments는 이름에 값을 할당하는 데 사용됩니다. 인수는 블록의 일부입니다. 일부 블록에는 필수 인자와 몇 가지 선택 인자가 있습니다.

```
resource “aws_instance" “example" {
  # EC2 instance resource
  instance_type                = “t2.micro”
  ami                          = “ami-zbc123ef”

  tags = {
    Name = “demo-ec2”
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

HCL syntax

Blocks | Arguments | **Identifilers** | Expressions | Comments

Identifiers 는 인자, 블록 유형 또는 테라폼 관련 구조체의 이름입니다. 식별자의 명명 규칙은 문자, 밑줄, 하이픈, 숫자를 포함할 수 있지만 숫자로 시작할 수 없습니다.

```
resource “aws_instance" “example" {
  # EC2 instance resource
  instance_type                = “t2.micro”
  ami                          = “ami-zbc123ef”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resources

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- 리소스는 인프라 구성 요소를 정의하는 코드 블록 입니다.
  예) aws_instance, aws_vpc

- Terraform은 리소스 유형과 리소스 이름을 사용하여 인프라 요소를 식별합니다.

```
resource “resource_type" “resource_name" {
  # Resource specific arguments
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Example for resources

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- resource 키워드는 블록을 클라우드 인프라 구성 요소로 식별하는 데 사용됩니다.

- 리소스 유형 : aws_s3_bucket
- 리소스 이름 : example-bucket

```
resource “aws_s3_bucket" “example-bucket" {
  bucket                  = “<unique-bucket-name>”
  acl                     = “private”

  versioning {
    enable                = true
  }

  tags = {
    Name                  = “example-bucket”
    Environment           = “Dev”
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resource arguments

```
-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf
```

- arguments는 리소스 유형에 따라 다릅니다.
- 일부 인수는 필수이고 다른 인수는 선택 사항입니다.
  필수 : ami, instance_type
  선택 : user_data, tag

```
# Create an EC2 instance
resource "aws_instance" "example" {
  ami                      = "ami-785db401"
  instance_type            = "t2.micro"
  vpc_security_group_ids   = ["${aws_security_group.instance.id}"]

  user_data = <<-EOF
              #!/bin/bash
              echo "Hello, World" > index.html
              nohup busybox httpd -f -p 8080 &
              EOF

  tags {
    Name = "terraform-example"
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Provider(1 of 2)

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- Terraform은 providers가 선언 될 때 루트 구성에서 providers 플러그인을 다운로드 합니다.

- 프로바이더는 특정 API를 Terraform 리소스로 노출하고 그 상호작용을 관리합니다.

```
terraform {
  required_providers {
    aws = {
      source = “hashicorp/aws”
      version = “~> 5.0”
    }
  }
}

provider “aws” {
  region  = “us-east-1”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Provider(2 of 2)

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- 프로바이더 구성은 Terraform 구성의 루트 모듈에서 선언 합니다.
- region 과 같은 인수는 공급자 블록 내에서 선언할 수 있습니다.

```
terraform {
  required_providers {
    aws = {
      source = “hashicorp/aws”
      version = “~> 5.0”
    }
  }
}

provider “aws” {
  region  = “us-east-1”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Provider versions

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- version 인수는 선택 사항이지만 권장됩니다.
- 버전 인수는 공급자를 특정 버전 또는 버전 범위로 제한하는 데 사용됩니다.

```
terraform {
  required_providers {
    aws = {
      source = “hashicorp/aws”
      version = “~> 5.0”
    }
  }
}

provider “aws” {
  region  = “us-east-1”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variables

리소스 인수를 매개변수화하여 값의 하드 코딩을 제거합니다.

예) region, AZs, AMI 등

run time 의 리소스 속성을 정의하거나 확장자가 .tfvars인 파일을 이용해서 중앙에서 정의합니다.

aws_instance

main.tf

aws_instance
선언

ami

instance_type

tags

variables.tf

리소스 속성
매개 변수화

```
variable “ami” {
  description = “test”
  type = string
  default = null
}
```

terraform.tfvars

run time 또는
파일에 값을 정의

```
ami= “ami-123dafdes32”
instance_type=“t3.micro”
```

CLI

```
$terraform apply

ami= “ami-123dafdes32”
instance_type=“t3.micro”
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variables

Variables.tf

```
variable "image_id" {
  type = string
}

variable
"availability_zone_names" {
  type    = list(string)
  default = ["us-west-1a"]
}

variable "ami_id_maps" {
  type = map
  default = {}
}
```

Terraform.tfvars

```
image_id = "ami-064c81ce3a290fde1"
availability_zone_names = ["us-west-1a","us-west-
1b","us-west-1c"]
ami_id_maps = {
    ap-northeast-2 = {
      amazon_linux2 = "ami-010bf43fe22f847ed"
      ubuntu_18_04  = "ami-061b0ee20654981ab"
    }

    us-east-1 = {
      amazon_linux2 = "ami-0d29b48622869dfd9"
      ubuntu_18_04  = "ami-0d324124b7b7eec66"
    }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Outputs values

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf

- Outputs 값은 output.tf 파일에 저장 됩니다.
- 출력 값은 리소스 속성의 값을 노출 합니다.

```
output "public_ip" {
  value = aws_instance.webserver.public_ip
}
```

```
#terraform apply
aws_instance.webserver: Creating…
aws_instance.webserver: Creating compute after ls []
Apply complete! Resources: 1 added, 0 changed, 0 destoryed.
Outputs:
public_ip = "35.212.5.123"
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

State

-- main.tf
-- providers.tf
-- variables.tf
-- outputs.tf
-- terraform.tfvars
-- terraform.tfstate

- Terraform은 관리하는 리소스의 상태를 상태 파일에 저장합니다.
- 상태 파일은 다음 내용을 저장 합니다.
  - ✓ Locally (default)
  - ✓ Remotely in a shared location

You do not modify this file.


```
{
  "version": 4,
  "terraform_version": "1.0.11",
  "serial": 3,
  "lineage": "822c3d96-0500-29cd-68e3-13101f2846f0",
  "outputs": {
    "vm_name": {
      "value": "terraform-test",
      "type": "string"
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": “aws_vpc”,
      “name”: “myvpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
```


Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

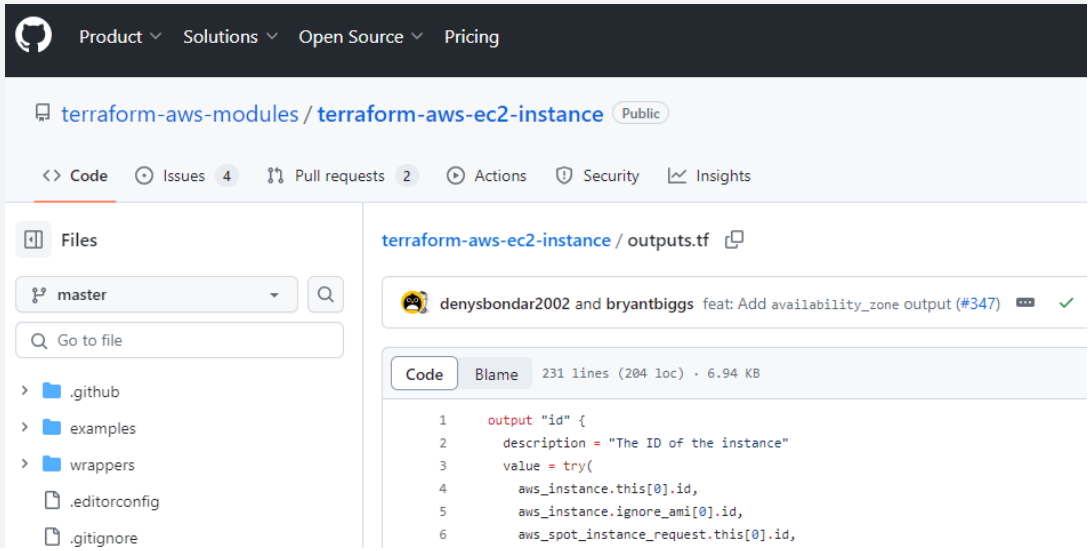
Modules

Terraform 모듈은 단일 디렉토리에 있는 Terraform 구성 파일의 집합니다.

- 모듈은 Terraform에서 코드 재사용을 위한 주요 방법입니다.
- Source에는 두가지 종류가 있습니다.
  - ➢ Local : 디렉토리 내의 소스
  - ➢ Remote : 디렉토리 외부 소스

```
-- instances/
   -- main.tf
   -- variables.tf
   -- outputs.tf
   -- terraform.tfvars
```

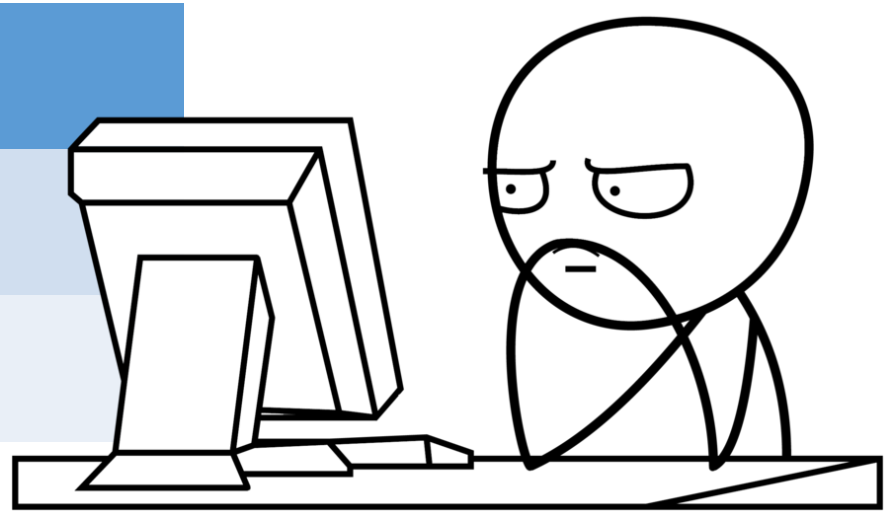
module



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01	The Author phase(작성 단계)
02	Terraform commands
03	Terraform Validator Tool



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Terraform 명령어

terraform init	지정한 공급자의 플러그을 이용하여 초기화 작업 진행
terraform plan	Terraform apply 후 생성될 리소스 미리보기
terraform apply	실제 인프라 리소스를 생성합니다
terraform destory	인프라 리소스를 삭제합니다
terraform fmt	Terraform 표준 규칙에 맞게 자동으로 형식을 지정합니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Initialize Terraform using terraform init

Scope	Author	Initialize	Plan	Scope
Project 필요 리소스 확인	Scope를 베이스로 구성 파일 작성	Provider plugins 다운로드 및 디렉토리 초기화	리소스 실행 계획 확인 (생성,수정,소멸)	실제 인프라 리소스를 생성.

Terraform Workflow

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

초기화(init) 단계:

terraform init provider plugins 다운로드

-- main.tf
    -- servers/
        -- main.tf
        -- variables.tf
            --

outputs.tf

```
terraform {
  required_providers {
    aws = {
      source = “hashicorp/aws”
    }
  }
}
```

```
$ terraform init
Initializing the backend…

Initializing provider plugins…
-    Finding latest version of hashicorp/aws…
-    Installing hashicorp/aws v4.21.0…
-    Installed hashicorp/aws v4.21.0 (signed by HashiCorp)
..
Terrform has been successfully initialized!
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Preview resource action using terraform plan

Scope	Author	Initialize	Plan	Apply
Project 필요 리소스 확인	Scope를 베이스로 구성 파일 작성	Provider plugins 다운로드 및 디렉토리 초기화	리소스 실행 계획 확인 (생성,수정,소멸)	실제 인프라 리소스를 생성.

Terraform Workflow

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

계획(Plan) 단계:

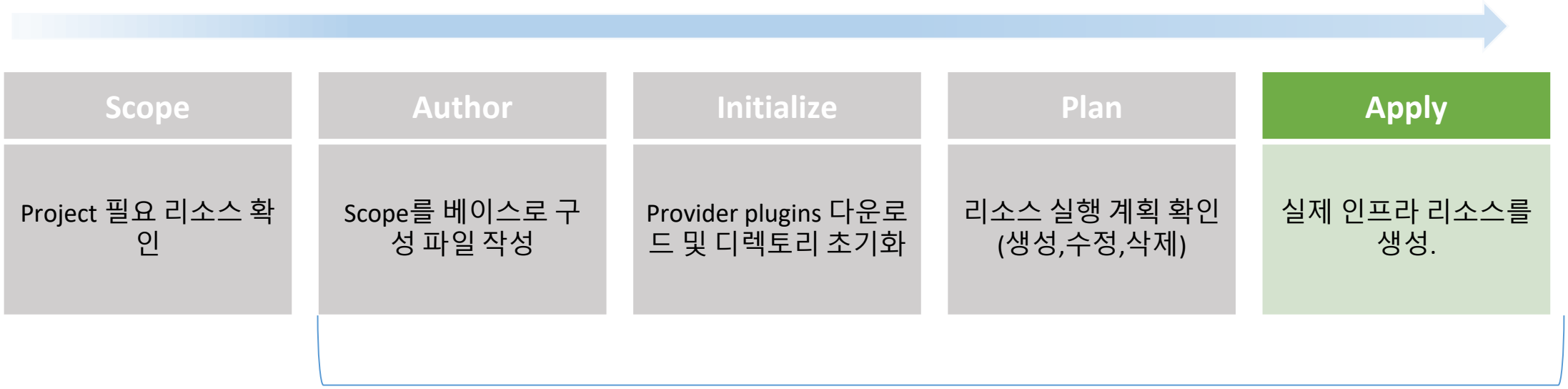
terraform plan 실행 계획(Plan)을 생성

```
resource “aws_instance” “public_ec2” {
  ami             = “ami-0db78afd3d150fc18”
  instance_type   = “t3.small”
}
```

```
$terraform plan
Terraform will perform the following actions:
  # module.ec2.aws_instance.public_ec2 will be created
  + resource "aws_instance" "public_ec2" {
      + ami = "ami-0db78afd3d150fc18"
      + arn = (known after apply)
      + associate_public_ip_address = (known after apply)
      + instance_type = "t3.small“
}
Plan: 1 to add, 0 to change, 0 to destroy.
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform apply를 사용하여 Terraform plan에 제안된 작업을 실행



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

적용(Apply) 단계:

terraform apply 계획을(plan) 적용 즉 실행 한다.

```
resource “aws_instance” “public_ec2” {
  ami             = “ami-0db78afd3d150fc18”
  instance_type   = “t3.small”
}
```

```
$terraform apply
Terraform will perform the following actions:
  # module.ec2.aws_instance.public_ec2 will be created
  + resource "aws_instance" "public_ec2" {
      + ami = "ami-0db78afd3d150fc18"
      + arn = (known after apply)
      + associate_public_ip_address = (known after apply)
      + instance_type = "t3.small“
} Apply changes: yes
aws_instance.public_ec2: Creating...
aws_instance.public_ec2 : Creation complete after 1s [id= student03342a2334524]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Code convertions

HCL 형식(Formatting) 베스트 사례

- 메타인수(meta arguments)다른 인수와 구분 합니다.
- 들여쓰기(indent)는 두 개의 공백을 사용합니다.
- 등호(=)에서 값을 정렬합니다.
- 중첩된 블록을 인수 아래에 배치
- 블록을 빈 줄로 구분

구성 언어 - Style Conventions

```
resource “aws_instance” “mzc-tf-vm01” {
     tags { #nested arguments above
       Name = “webserver-1”
     }
     count = 2 #meta-argument in between
     instance_type = “t2.micro” #unaligned equal signs
     ami-04e914639d0cca79a
     ..
}

resource “google_compute_instance” “mzc-tf-vm01” {
     count                    = 2 #meta-argument last

     ami                      = “ami-04e914639d0cca79a”
     instance_type            = “t2.micro” #align equal signs
     tags { #nested arguments below
       Name                   = “webserver-1”
     }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform fmt

Terraform fmt는 코드 규칙 모범 사례를 적용할 수 있습니다.

가독성과 일관성을 위해 자동으로 구성을 업데이트 합니다.

terraform fmt 적용 전:

```
resource “aws_instance” “mzc-tf-vm01” {
     tags { #nested arguments above
       Name = “webserver-1”
     }
     count = 2 #meta-argument in between
     instance_type=t2.micro #unaligned equal signs
     ami-04e914639d0cca79a
     ..
}
```

terraform fmt 적용 후:

```
resource “google_compute_instance” “mzc-tf-vm01” {
     count                    = 2 #meta-argument last

     ami                      = “ami-04e914639d0cca79a”
     instance_type            = t2.micro #align equal signs
     tags { #nested arguments below
       Name                   = “webserver-1”
     }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform destory command

```
$ terraform destroy
aws_s3_bucket.b: Refreshing state... [id=mybucket-seyslee-terraform-training]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:
  # aws_s3_bucket.b will be destroyed
  - resource "aws_s3_bucket" "b" {
      - arn                         = "arn:aws:s3:::mybucket-seyslee-terraform-training" -
      - bucket                      = "mybucket-seyslee-terraform-training" -> null
      - bucket_domain_name          = "mybucket-seyslee-terraform-training.s3.amazonaws.co
     ...
    }
Plan: 0 to add, 0 to change, 1 to destroy.
..
s3.mybucket: Destroying... [id= mybucket-seyslee-terraform-training]
s3.mybucket: Destruction complete after 0s
…
Destroy complete! Resources: 1 destroyed.
```

버킷에 데이터가 있는 경우, 해당 데이터는 복구할 수 없으므로 테라폼 파기를 실행할 때 주의

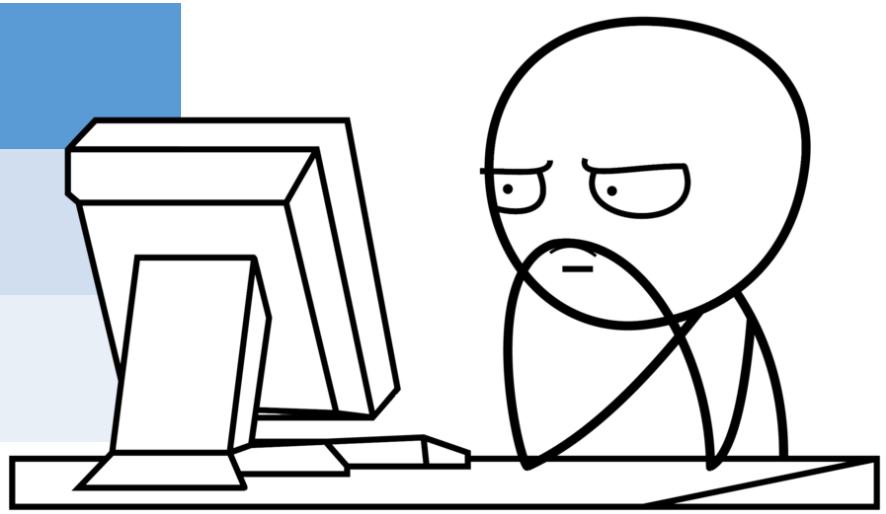
Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 The Author phase(작성 단계)

02 Terraform commands

03 **Terraform Validator Tool**



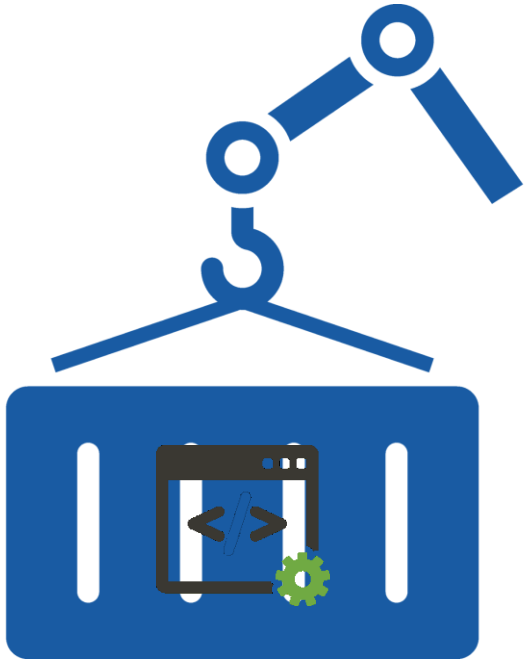
Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

유효성(validate) phase

유효성(validate)
tfsec, TFLint 및 checkov를 포함한 IaC 유효성 검사 도구를 실행 그외 terraform validate 및 terraform fmt

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

유효성(validate) 도구 사용 이유



보안 및 거버넌스 팀
이 제약조건 설정

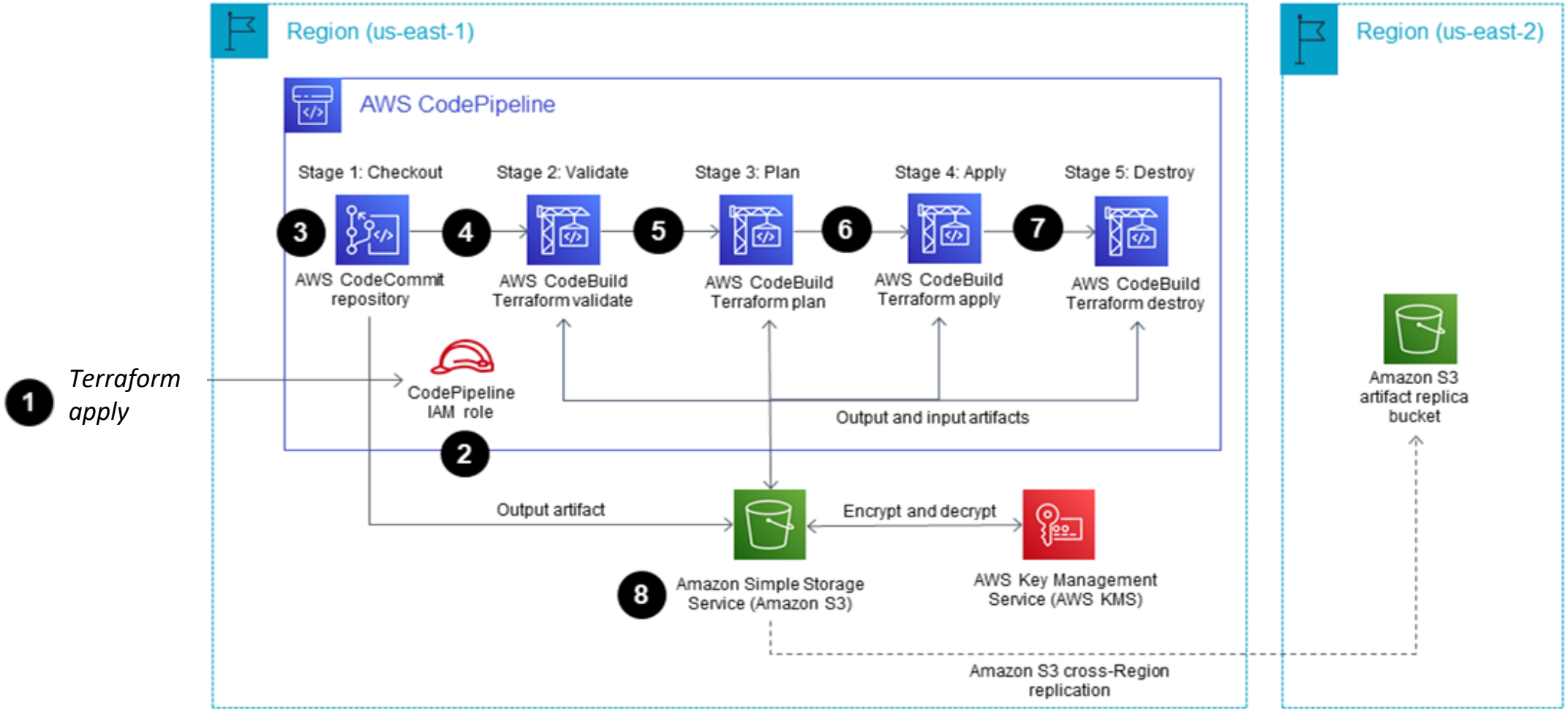
Automate validation

거버넌스 정책

보안 유효성

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

AWS Code Pipeline을 사용한 테리폼 구성 검증



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Terraform Validator uses

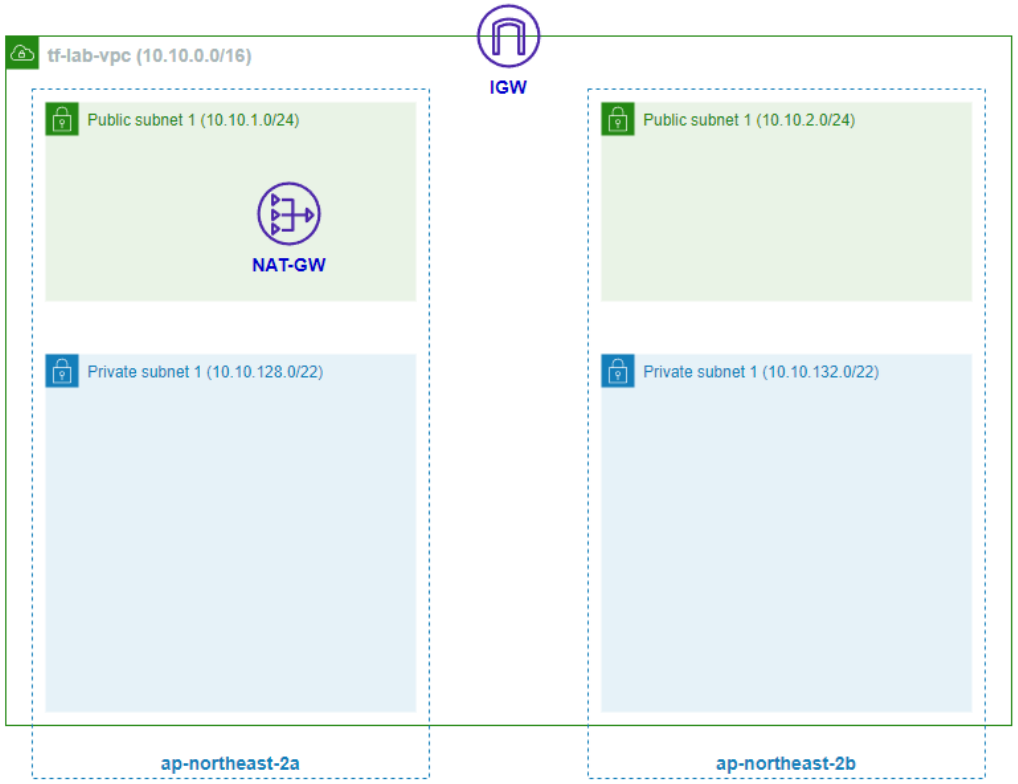
01	Platform 팀은 인프라 CI/CD 파이프라인에 가이드라인을 추가하여 모든 변경 사항의 유효성을 검사 할 수 있습니다.
02	Application 팀과 developers 는 조직 중앙 정책 라이브러리를 사용하여 Terraform 구성을 검증 합니다.
03	보안팀은 중앙 집중식 정책 라이브러리를 생성하여 정책 위반을 식별하고 방지할 수 있습니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Lab : AWS VPC 기본 구성 및 Terraform 명령 사용

실습시간 : 60min

- **실습 세부 내용**
  - 실습 디렉터리 및 파일 생성 /Lab02/main.tf
  - aws_vpc 생성
    - subnet
    - Internet gateway
    - Route Table
    - Private subnet and NAT gateway



Public Route Table

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	IGW

Private Route Table 1

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	NAT-GW

Private Route Table 2

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	NAT-GW

Thank you